In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error , mean_squared_error , r2_score

import warnings
warnings.filterwarnings('ignore')
```

In [5]:
```
df = pd.read_csv(r"C:\Users\basam\CAR DETAILS FROM CAR DEKHO.csv")
df.head()
```

Out[5]:

	name	year	selling_price	km_driven	fuel	seller_type	transmission	owner
0	Maruti 800 AC	2007	60000	70000	Petrol	Individual	Manual	First Owner
1	Maruti Wagon R LXI Minor	2007	135000	50000	Petrol	Individual	Manual	First Owner
2	Hyundai Verna 1.6 SX	2012	600000	100000	Diesel	Individual	Manual	First Owner
3	Datsun RediGO T Option	2017	250000	46000	Petrol	Individual	Manual	First Owner
4	Honda Amaze VX i-DTEC	2014	450000	141000	Diesel	Individual	Manual	Second Owner

In [7]:
```
df.shape
```

Out[7]:
```
(4340, 8)
```

In [8]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4340 entries, 0 to 4339
Data columns (total 8 columns):
 #   Column         Non-Null Count  Dtype 
---  ------         --------------  ----- 
 0   name           4340 non-null   object
 1   year           4340 non-null   int64 
 2   selling_price  4340 non-null   int64 
 3   km_driven      4340 non-null   int64 
 4   fuel           4340 non-null   object
 5   seller_type    4340 non-null   object
 6   transmission   4340 non-null   object
 7   owner          4340 non-null   object
dtypes: int64(3), object(5)
memory usage: 271.4+ KB
```

In [31]:
```
df.columns
```

Out[31]:
```
Index(['year', 'selling_price', 'km_driven', 'fuel', 'seller_type',
       'transmission', 'owner'],
      dtype='object')
```

In [9]:
```
df.describe()
```

Out[9]:

	year	selling_price	km_driven
count	4340.000000	4.340000e+03	4340.000000
mean	2013.090783	5.041273e+05	66215.777419
std	4.215344	5.785487e+05	46644.102194
min	1992.000000	2.000000e+04	1.000000
25%	2011.000000	2.087498e+05	35000.000000
50%	2014.000000	3.500000e+05	60000.000000
75%	2016.000000	6.000000e+05	90000.000000
max	2020.000000	8.900000e+06	806599.000000

In [10]:
```
df.isnull().sum()
```

Out[10]:
```
name             0
year             0
selling_price    0
km_driven        0
fuel             0
seller_type      0
transmission     0
owner            0
dtype: int64
```

In [11]:
```
df.duplicated().sum()
```

Out[11]: np.int64(763)

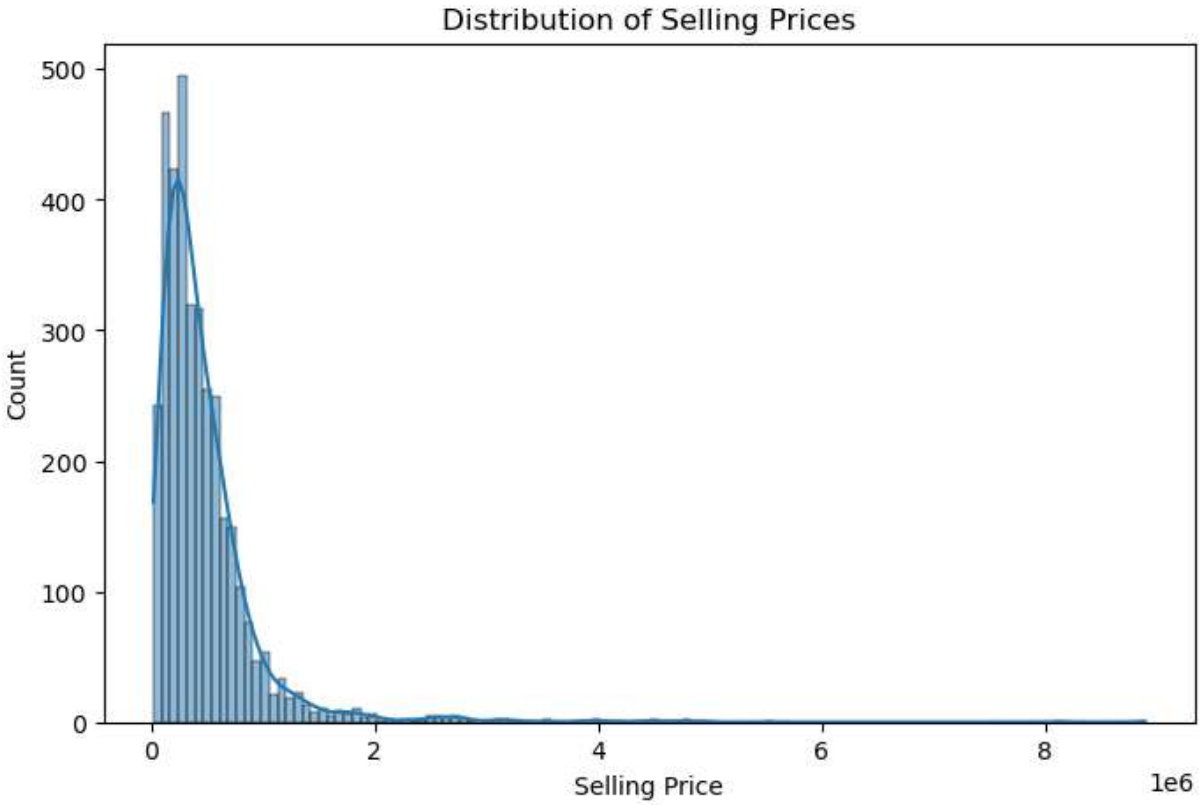
In [12]:
```
df = df.drop_duplicates()
df.shape
```

Out[12]: (3577, 8)

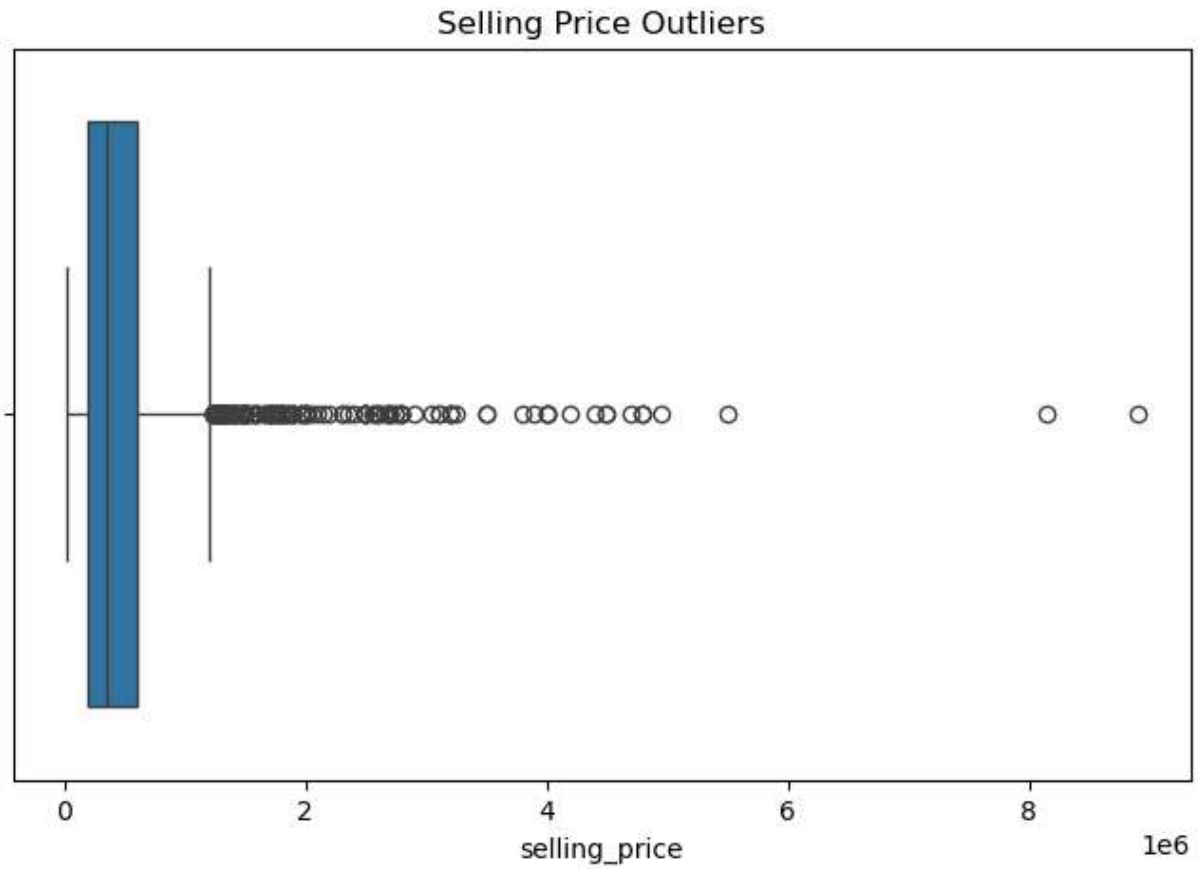
In [30]:
```
df.dtypes
```

Out[30]:
```
year              int64
selling_price     int64
km_driven         int64
fuel             object
seller_type      object
transmission     object
owner            object
dtype: object
```

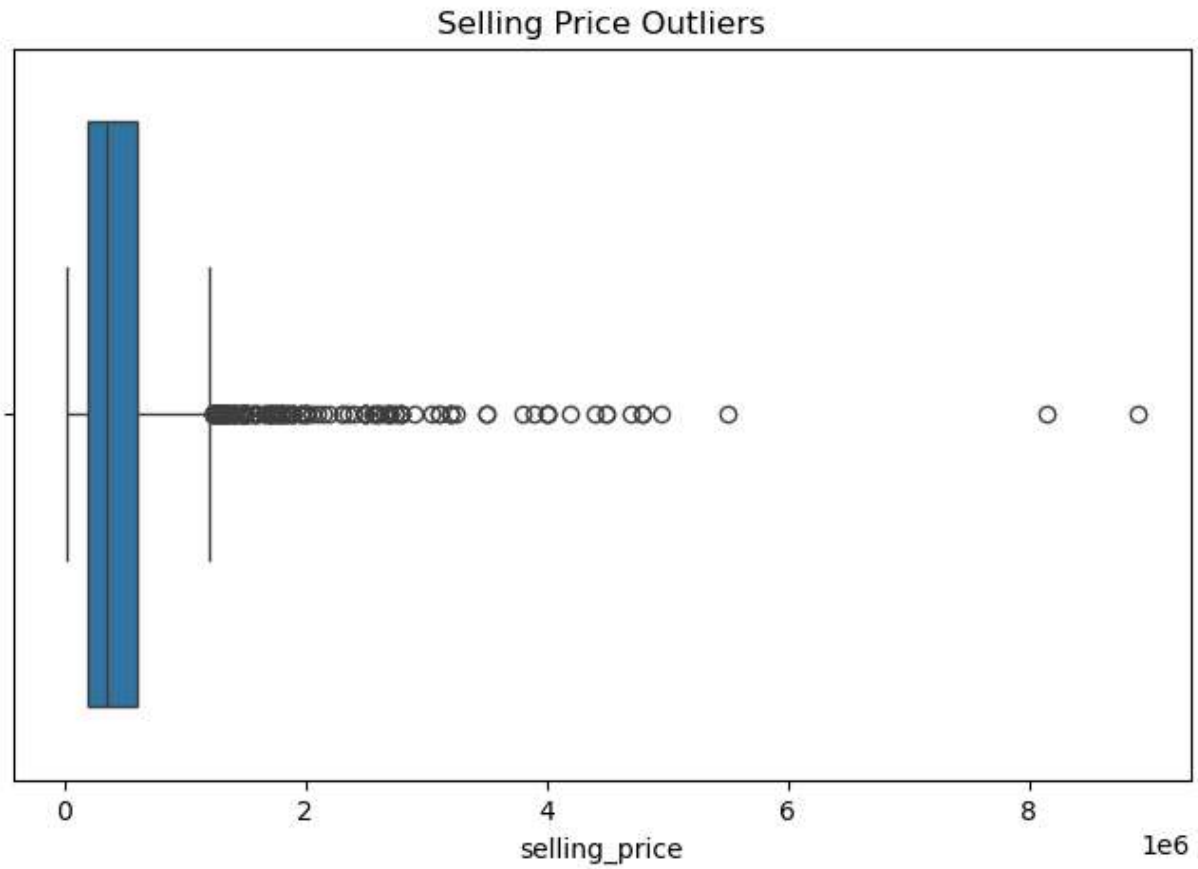
In [14]:
```
plt.figure(figsize=(8, 5))
sns.histplot(data=df, x="selling_price", kde=True)
plt.title("Distribution of Selling Prices")
plt.xlabel("Selling Price")
plt.show()
```



In [15]:
```
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x="selling_price")
plt.title("Selling Price Outliers")
plt.show()
```



In [16]:
```
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x="selling_price")
plt.title("Selling Price Outliers")
plt.show()
```



In [17]:
```
df.columns = df.columns.str.strip().str.lower()
df = df.drop_duplicates()
df = df.dropna()
df = df.drop(columns=["name"], errors="ignore")
df.head()
```

Out[17]:

	year	selling_price	km_driven	fuel	seller_type	transmission	owner
0	2007	60000	70000	Petrol	Individual	Manual	First Owner
1	2007	135000	50000	Petrol	Individual	Manual	First Owner
2	2012	600000	100000	Diesel	Individual	Manual	First Owner
3	2017	250000	46000	Petrol	Individual	Manual	First Owner
4	2014	450000	141000	Diesel	Individual	Manual	Second Owner

In [18]:
```
X = df.drop(columns=["selling_price"])
y = df["selling_price"]
```

In [19]:
```
X = pd.get_dummies(X, drop_first=True, dtype=int)

X.head()
```

Out[19]:

	year	km_driven	fuel_Diesel	fuel_Electric	fuel_LPG	fuel_Petrol	seller_type_Individual
0	2007	70000	0	0	0	1	1
1	2007	50000	0	0	0	1	1
2	2012	100000	1	0	0	0	1
3	2017	46000	0	0	0	1	1
4	2014	141000	1	0	0	0	1

In [20]:
```
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.20,random_state
X_train.shape, X_test.shape
```

Out[20]:
```
((2861, 13), (716, 13))
```

In [21]:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [22]:
```
linear_model = LinearRegression()
linear_model.fit(X_train_scaled, y_train)

tree_model = DecisionTreeRegressor(
    max_depth=10,
    random_state=42
)
tree_model.fit(X_train, y_train)

forest_model = RandomForestRegressor(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)
forest_model.fit(X_train, y_train)
```

Out[22]:

▾ RandomForestRegressor

▶ Parameters

In [23]:
```
linear_pred = linear_model.predict(X_test_scaled)
tree_pred = tree_model.predict(X_test)
forest_pred = forest_model.predict(X_test)
```

In [24]:
```
def evaluate_model(name, actual, predicted):
    mse = mean_squared_error(actual, predicted)

    return {
        "Model": name,
```

```
        "MAE": mean_absolute_error(actual, predicted),
        "MSE": mse,
        "RMSE": np.sqrt(mse),
        "R2 Score": r2_score(actual, predicted)
    }


results = pd.DataFrame([
    evaluate_model("Linear Regression", y_test, linear_pred),
    evaluate_model("Decision Tree", y_test, tree_pred),
    evaluate_model("Random Forest", y_test, forest_pred)
])

results.round(3)
```

Out[24]:

	Model	MAE	MSE	RMSE	R2 Score
0	Linear Regression	213071.177	1.956337e+11	442304.969	0.393
1	Decision Tree	202374.135	2.198864e+11	468920.484	0.317
2	Random Forest	193601.410	1.795643e+11	423750.332	0.443

In [25]:
```
best_model_name = results.loc[results["R2 Score"].idxmax(), "Model"]

predictions = {
    "Linear Regression": linear_pred,
    "Decision Tree": tree_pred,
    "Random Forest": forest_pred
}

best_predictions = predictions[best_model_name]

best_model_name
```

Out[25]:
```
'Random Forest'
```

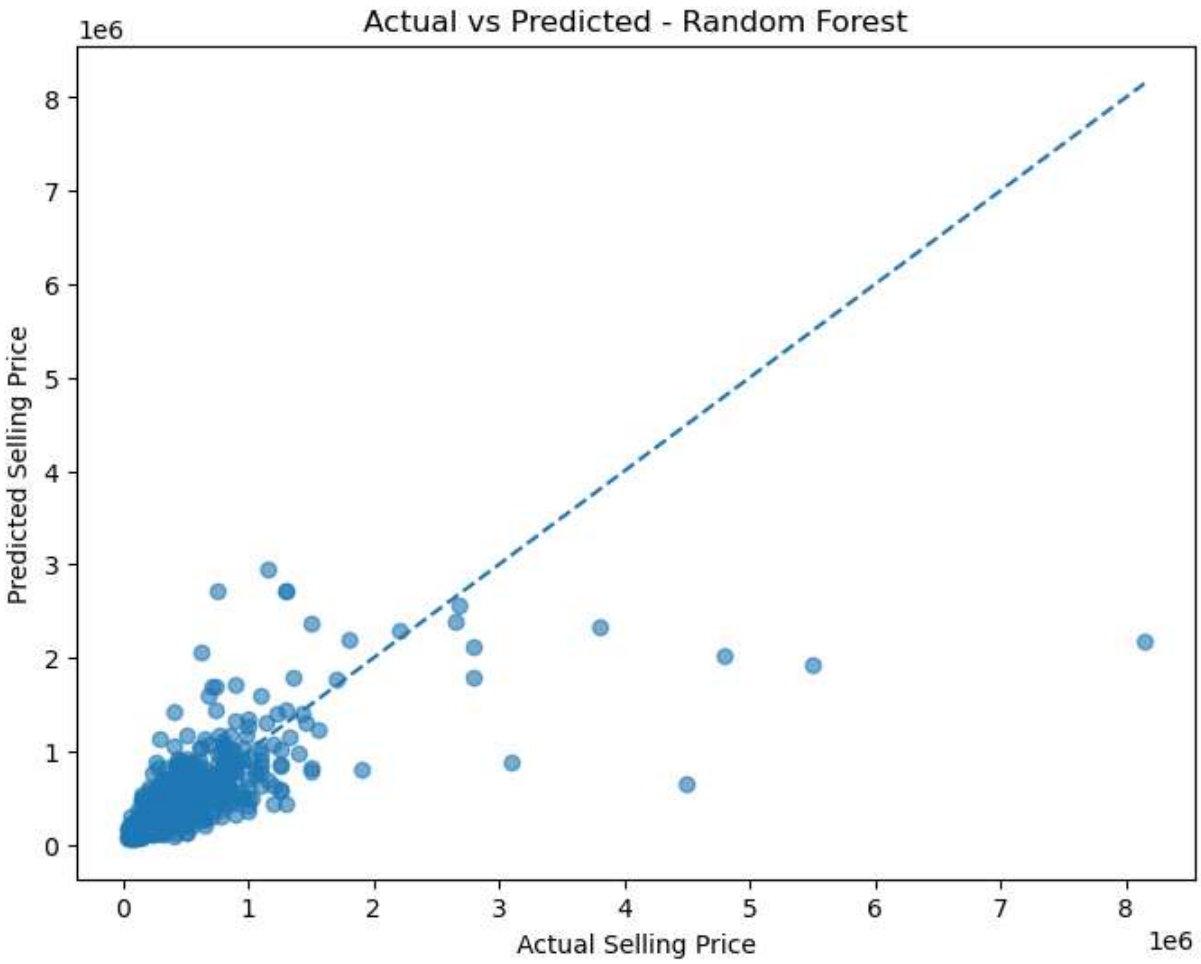
In [26]:
```
plt.figure(figsize=(8, 6))

plt.scatter(y_test, best_predictions, alpha=0.6)

minimum = min(y_test.min(), best_predictions.min())
maximum = max(y_test.max(), best_predictions.max())

plt.plot(
    [minimum, maximum],
    [minimum, maximum],
    linestyle="--"
)

plt.xlabel("Actual Selling Price")
plt.ylabel("Predicted Selling Price")
plt.title("Actual vs Predicted - " + best_model_name)
plt.show()
```



In [27]:
```
importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": forest_model.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

importance.head(10)
```

Out[27]:

	Feature	Importance
0	year	0.276128
1	km_driven	0.275607
8	transmission_Manual	0.255870
2	fuel_Diesel	0.080052
5	fuel_Petrol	0.037555
6	seller_type_Individual	0.034694
10	owner_Second Owner	0.023139
11	owner_Test Drive Car	0.008627
12	owner_Third Owner	0.004689
7	seller_type_Trustmark Dealer	0.002580

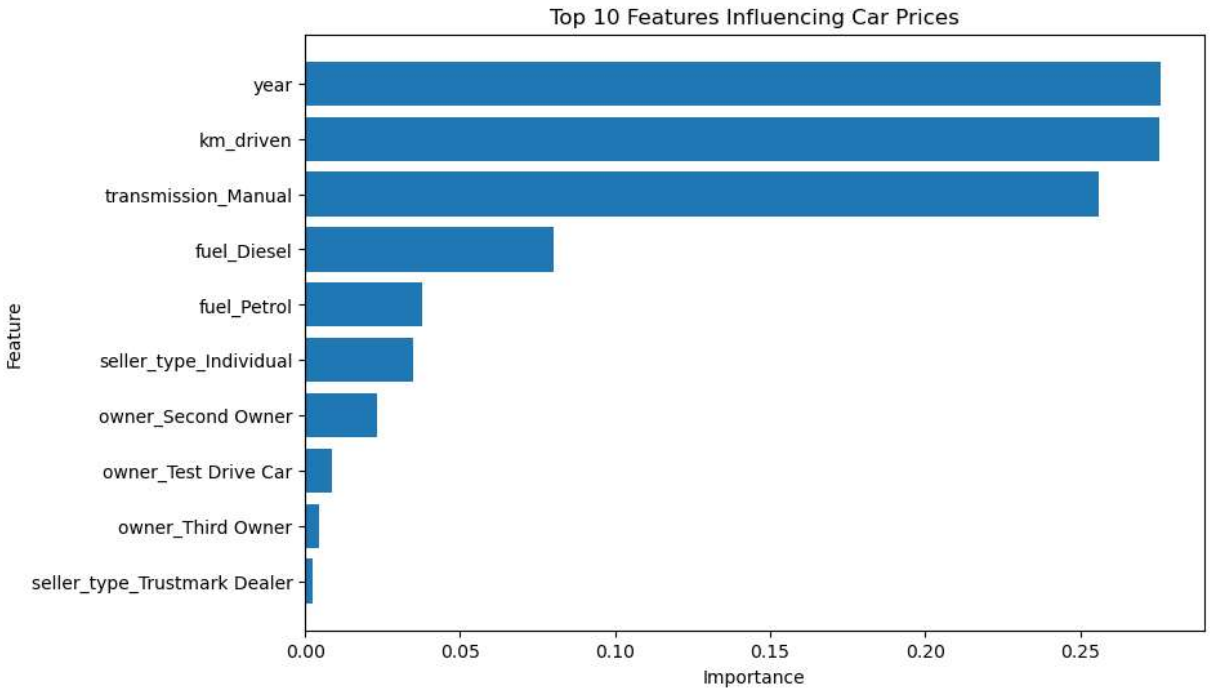
In [28]:

```
top_features = importance.head(10).sort_values(
    by="Importance"
)

plt.figure(figsize=(9, 6))

plt.barh(
    top_features["Feature"],
    top_features["Importance"]
)

plt.xlabel("Importance")
plt.ylabel("Feature")
plt.title("Top 10 Features Influencing Car Prices")
plt.show()
```



In [29]:
```
best_result = results.loc[results["R2 Score"].idxmax()]

{
    "Best Model": best_model_name,
    "MAE": round(best_result["MAE"], 2),
    "MSE": round(best_result["MSE"], 2),
    "RMSE": round(best_result["RMSE"], 2),
    "R2 Score": round(best_result["R2 Score"], 3)
}
```

Out[29]:
```
{'Best Model': 'Random Forest',
 'MAE': np.float64(193601.41),
 'MSE': np.float64(179564343673.75),
 'RMSE': np.float64(423750.33),
 'R2 Score': np.float64(0.443)}
```

In [ ]: